

---

# How Spark Works

This chapter introduces Spark's place in the big data ecosystem and its overall design. Spark is often considered an alternative to Apache MapReduce, since Spark can also be used for distributed data processing with Hadoop. <sup>1</sup>, packaged with the distributed file system Apache Hadoop.] As we will discuss in this chapter, Spark's design principals are quite different from MapReduce's and Spark does not need to be run in tandem with Apache Hadoop. Furthermore, while Spark has inherited parts of its API, design, and supported formats from existing systems, particularly DryadLINQ, Spark's internals, especially how it handles failures, differ from many traditional systems. <sup>2</sup> Spark's ability to leverage lazy evaluation within memory computations make it particularly unique. Spark's creators believe it to be the first high-level programming language for fast, distributed data processing. <sup>3</sup> Understanding the general design principals behind Spark will be useful for understanding the performance of Spark jobs.

To get the most out of Spark, it is important to understand some of the principles used to design Spark and, at a cursory level, how Spark programs are executed. In this chapter, we will provide a broad overview of Spark's model of parallel computing and

---

<sup>1</sup> MapReduce is a programmatic paradigm that defines programs in terms of *map* procedures that filter and sort data onto the nodes of a distributed system, and *reduce* procedures that aggregate the data on the mapper nodes. Implementations of MapReduce have been written in many languages, but the term usually refers to a popular implementation called link::http://hadoop.apache.org/[*Hadoop MapReduce*

<sup>2</sup> DryadLINQ is a Microsoft research project which puts the .NET Language Integrated Query (LINQ) on top of the Dryad distributed execution engine. Like Spark, The DryadLINQ API defines an object representing a distributed dataset and exposes functions to transform data as methods defined on the dataset object. DryadLINQ is lazily evaluated and its scheduler is similar to Spark's however, it doesn't use in memory storage. For more information see [the DryadLINQ documentation](#).

<sup>3</sup> See [the original Spark Paper](#).

a thorough explanation of the Spark scheduler and execution engine. We will refer to the concepts in this chapter throughout the text. Further, this explanation will help you get a more precise understanding of some of the terms you've heard tossed around by other Spark users and in the Spark documentation.

## How Spark Fits into the Big Data Ecosystem

Apache Spark is an open source framework that provides highly generalizable methods to process data in parallel. On its own, Spark is not a data storage solution. Spark can be run locally, on a single machine with a single JVM (called local mode). More often Spark is used in tandem with a distributed storage system to write the data processed with Spark (such as HDFS, Cassandra, or S3) and a cluster manager to manage the distribution of the application across the cluster. Spark currently supports three kinds of cluster managers: the manager included in Spark, called the Standalone Cluster Manager, which requires Spark to be installed in each node of a cluster, Apache Mesos; and Hadoop YARN.

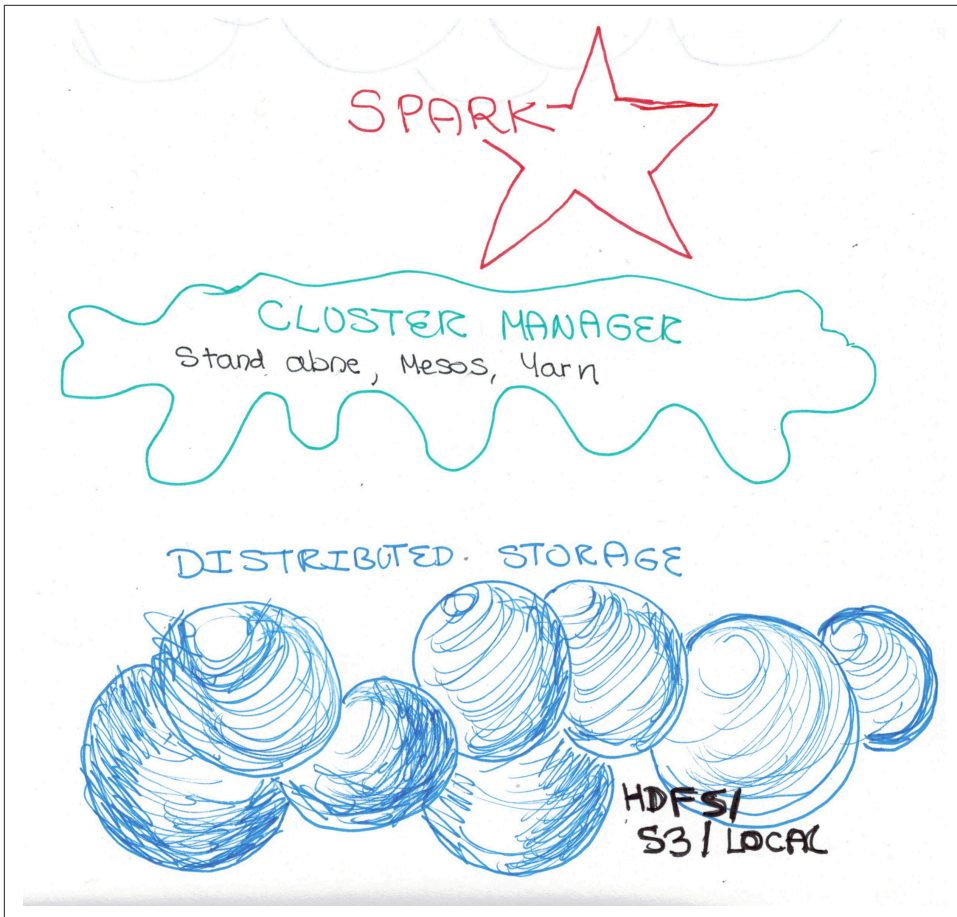


Figure 2-1. A diagram of the data processing ecosystem including Spark.

## Spark Components

Spark provides a high-level query language to process data. Spark Core, the main data processing framework in the Spark ecosystem, has APIs in Scala, Java, and Python. Spark is built around a data abstraction called *Resilient Distributed Datasets* (RDDs). RDDs are a representation of lazily evaluated statically typed distributed collections. RDDs have a number of predefined “coarse grained” transformations (transformations that are applied to the entire dataset), such as `map`, `join`, and `reduce`, as well as I/O functionality, to move data in and out of storage or back to the driver.

In addition to Spark Core, the Spark ecosystem includes a number of other first-party components for more specific data processing tasks, including Spark SQL, Spark MLlib, Spark ML, and Graph X. These components have many of the same generic

performance considerations as the core. However, some of them have unique considerations - like SQL's different optimizer.

*Spark SQL* is a component that can be used in tandem with the Spark Core. Spark SQL defines an interface for a semi-structured data type, called `DataFrames` and a typed version called `Dataset`, with APIs in Scala, Java, and Python, as well as support for basic SQL queries. Spark SQL is a very important component for Spark performance, and much of what can be accomplished with Spark core can be applied to Spark SQL, so we cover it deeply in [Chapter 3](#).

Spark has two machine learning packages, ML and MLlib. MLlib, one of Spark's machine learning components is a package of machine learning and statistics algorithms written with Spark. Spark ML is still in the early stages, but since Spark 1.2, it provides a higher-level API than MLlib that helps users create practical machine learning pipelines more easily. Spark MLlib is primarily built on top of RDDs, while ML is built on top of SparkSQL data frames.<sup>4</sup> Eventually the Spark community plans to move over to ML and deprecate MLlib. Spark ML and MLlib have some unique performance considerations, especially when working with large data sizes and caching, and we cover some of these in [???](#).

Spark Streaming uses the scheduling of the Spark Core for streaming analytics on mini batches of data. Spark Streaming has a number of unique considerations such as the window sizes used for batches. We offer some tips for using Spark Streaming in [???](#).

Graph X is a graph processing framework built on top of Spark with an API for graph computations. Graph X is one of the least mature components of Spark, so we don't cover it in much detail. In future version of Spark, typed graph functionality will start to be introduced on top of the `Dataset` API. We will provide a cursory glance at Graph X in [???](#).

This book will focus on optimizing programs written with the Spark Core and Spark SQL. However, since MLlib and the other frameworks are written using the Spark API, this book will provide the tools you need to leverage those frameworks more efficiently. Who knows, maybe by the time you're done, you will be ready to start contributing your own functions to MLlib and ML!

Beyond first party components, a large number of libraries both extend Spark for different domains and offer tools to connect it to different data sources. Many libraries are listed at <http://spark-packages.org/>, and can be dynamically included at runtime with `spark-submit` or the `spark-shell` and added as build dependencies to our

---

<sup>4</sup> See [The MLlib documentation](#).

maven or sbt project. We first use Spark packages to add support for csv data in “Additional Formats” on page 66 and then in more detail in ???

## Spark Model of Parallel Computing: RDDs

Spark allows users to write a program for the *driver* (or master node) on a cluster computing system that can perform operations on data in parallel. Spark represents large datasets as RDDs, immutable distributed collections of objects, which are stored in the *executors* or (slave nodes). The objects that comprise RDDs are called partitions and may be (but do not need to be) computed on different nodes of a distributed system. The Spark cluster manager handles starting and distributing the Spark executors across a distributed system according to the configuration parameters set by the Spark application. The Spark execution engine itself distributes data across the executors for a computation. See Figure 2-4.

Rather than evaluating each transformation as soon as specified by the driver program, Spark evaluates RDDs lazily, computing RDD transformations only when the final RDD data needs to be computed (often by writing out to storage or collecting an aggregate to the driver). Spark can keep an RDD loaded in memory on the executor nodes throughout the life of a Spark application for faster access in repeated computations. As they are implemented in Spark, RDDs are immutable, so transforming an RDD returns a new RDD rather than the existing one. As we will explore in this chapter, this paradigm of lazy evaluation, in memory storage and mutability allows Spark to be an easy-to-use as well as efficiently, fault-tolerant and general highly performant.

### Lazy Evaluation

Many other systems for in-memory storage are based on “fine grained” updates to mutable objects, i.e., calls to a particular cell in a table by storing intermediate results. In contrast, evaluation of RDDs is completely lazy. Spark does not begin computing the partitions until an action is called. An action is a Spark operation which returns something other than an RDD, triggering evaluation of partitions and possibly returning some output to a non-Spark system for example bringing data back to the driver (with operations like count or collect) or writing data to external storage system (such as copyToHadoop). Actions trigger the scheduler, which builds a *directed acyclic graph* (called the DAG), based on the dependencies between RDD transformations. In other words, Spark evaluates an action by working backward to define the series of steps it has to take to produce each object in the final distributed dataset (each partition). Then, using this series of steps called the execution plan, the scheduler computes the missing partitions for each stage until it computes the whole RDD.

## Performance & Usability Advantages of Lazy Evaluation

Lazy evaluation allows Spark to chain together operations that don't require communication with the driver (called transformations with one-to-one dependencies) to avoid doing multiple passes through the data. For example, suppose you have a program that calls a `map` and a `filter` function on the same RDD. Spark can look at each record once and compute both the `map` and the `filter` on each partition in the executor nodes, rather than doing two passes through the data, one for the `map` and one for the `filter`, theoretically reducing the computational complexity by half.

Spark's lazy evaluation paradigm is not only more efficient, it is also easier to implement the same logic in Spark than in a different framework like MapReduce, which requires the developer to do the work to consolidate her mapping operations. Spark's clever lazy evaluation strategy lets us be lazy and expresses the same logic in far fewer lines of code, because we can chain together operations with narrow dependencies and let the Spark evaluation engine do the work of consolidating them. Consider the classic word count example in which, given a dataset of documents, parses the text into words and then compute the count for each word. The word count example in MapReduce which is roughly fifty lines of `code` (excluding import statements) in Java compared to a program that provides the same functionality in Spark. A Spark implementation is roughly fifteen lines of code in Java and five in Scala. It can be found [on the apache website](https://the.apache.website). Furthermore if we were to filter out some “stop words” and punctuation from each document before computing the word count, this would require adding the filter logic to the mapper to avoid doing a second pass through the data. An implementation of this routine for MapReduce can be found here: <https://github.com/kite-sdk/kite/wiki/WordCount-Version-Three>. In contrast, we can modify the spark routine above by simply putting a filter step before we begin the code shown above and Spark's lazy evaluation will consolidate the map and filter steps for us.

*Example 2-1.*

```
def withStopWordsFiltered(rdd : RDD[String], illegalTokens : Array[Char],
  stopWords : Set[String]): RDD[(String, Int)] = {
  val tokens: RDD[String] = rdd.flatMap(_.split(illegalTokens ++ Array[Char](' '))).
    map(_.trim.toLowerCase)
  val words = tokens.filter(token =>
    !stopWords.contains(token) && (token.length > 0) )
  val wordPairs = words.map((_, 1))
  val wordCounts = wordPairs.reduceByKey(_ + _)
  wordCounts
}
```

## Lazy Evaluation & Fault Tolerance

Spark is fault-tolerant, because each partition of the data contains the dependency information needed to re-calculate the partition. Distributed systems, based on mutable objects and strict evaluation paradigms, provide fault tolerance by logging updates or duplicating data across machines. In contrast, Spark does not need to maintain a log of updates to each RDD or log the actual intermediary steps, since the RDD itself contains all the dependency information needed to replicate each of its partitions. Thus, if a partition is lost, the RDD has enough information about its lineage to recompute it, and that computation can be parallelized to make recovery faster.

## In Memory Storage and Memory Management

Spark's biggest performance advantage over MapReduce is in use cases involving repeated computations. Much of this performance increase is due to Spark's storage system. Rather than writing to disk between each pass through the data, Spark has the option of keeping the data on the executors loaded into memory. That way, the data on each partition is available in memory each time it needs to be accessed.

Spark offers three options for memory management: in memory deserialized data, in memory as serialized data, and on disk. Each has different space and time advantages.

1. *In memory as deserialized Java objects*: The most intuitive way to store objects in RDDs is as the deserialized Java objects that are defined by the driver program. This form of in memory storage is the fastest, since it reduces serialization time; however, it may not be the most memory efficient, since it requires the data to be as objects.
2. *As serialized data*: Using the Java serialization library, Spark objects are converted into streams of bytes as they are moved around the network. This approach may be slower, since serialized data is more CPU-intensive to read than deserialized data; however, it is often more memory efficient, since it allows the user to choose a more efficient representation for data than as Java objects and to use a faster and more compact serialization model, such as Kryo serialization. We will discuss this in detail in ???.
3. *On Disk*: Last, RDDs, whose partitions are too large to be stored in RAM on each of the executors, can be written to disk. This strategy is obviously slower for repeated computations, but can be more fault-tolerant for long strings of transformations and may be the only feasible option for enormous computations.

The `persist()` function in the RDD class lets the user control how the RDD is stored. By default, `persist()` stores an RDD as deserialized objects in memory, but the user can pass one of numerous storage options to the `persist()` function to control how the RDD is stored. We will cover the different options for RDD reuse in ???.

When persisting RDDs, the default implementation of RDDs evicts the least recently used partition (called LRU caching). However you can change this behavior and control Spark's memory prioritization with the `persistencePriority()` function in the RDD class. See ???.

## Immutability and the RDD Interface

Spark defines an RDD interface with the properties that each type of RDD must implement. These properties include the RDD's dependencies and information about data locality that are needed for the execution engine to compute that RDD. Since RDDs are statically typed and immutable, calling a transformation on one RDD will not modify the original RDD but rather return a new RDD object with a new definition of the RDD's properties.

RDDs can be created in two ways: (1) by transforming an existing RDD or (2) from a Spark Context, `SparkContext`. The Spark Context represents the connection to a Spark cluster and one running Spark application. The Spark Context can be used to create an RDD from a local Scala object (using the `makeRDD`, or `parallelize` methods) or by reading from stable storage (text files, binary files, a Hadoop Context, or a Hadoop file). DataFrames can be created by the Spark SQL equivalent to a Spark Context, `SQLContext` object, which can be created from a Spark Context.

Spark uses five main properties to represent an RDD internally. The three required properties are the list of partition objects, a function for computing an iterator of each partition, and a list of dependencies on other RDDs. Optionally, RDDs also include a partitioner (for RDDs of rows of key-value pairs represented as Scala tuples) and a list of preferred locations (for the HDFS file). Although, as an end user, you will rarely need these five properties and are more likely to use predefined RDD transformations, it is helpful to understand the properties and know how to access them for conceptualizing RDDs and for debugging. These five properties correspond to the following five methods available to the end user (you):

- `partitions()` Returns an array of the partition objects that make up the parts of the distributed dataset. In the case of an RDD with a partitioner, the value of the index of each partition will correspond to the value of the `getPartition` function for each key in the data associated with that partition.
- `iterator(p, parentIters)` Computes the elements of partition `p` given iterators for each of its parent partitions. This function is called in order to compute each of the partitions in this RDD. This is not intended to be called directly by the user, rather this is used by Spark when computing actions. Still, referencing the implementation of this function can be useful in determining how each partition of an RDD transformation is evaluated.

- `dependencies()` Returns a sequence of dependency objects. The dependencies let the scheduler know how this RDD depends on other RDDs. There are two kinds of dependencies: *Narrow Dependencies* (`NarrowDependency` objects), which represent partitions that depend on one or a small subset of partitions in the parent, and *Wide Dependencies* (`ShuffleDependency` objects), which are used when a partition can only be computed by rearranging all the data in the parent. We will discuss the types of dependencies in “[Wide vs. Narrow Dependencies](#)” on [page 26](#).
- `partitioner()` Returns a Scala option type that contains a `partitioner` object if the RDD has a function between datapoint and partitioner associated with it, such as a `hashPartitioner`. This function returns `None` for all RDDs that are not of type `tuple` (do not represent Key-value data). An RDD that represents an HDFS file (implemented in `NewHadoopRDD.scala`) has a `partitioner` for each block of the file. We will discuss partitioning in detail in [???](#).
- `preferredLocations(p)` Returns information about the data locality of a partition, `p`. Specifically, this function returns a sequence of strings representing some information about each of the nodes, where the split `p` is stored. In an RDD representing an HDFS file, each string in the result of `preferredLocations` is the Hadoop name of the node.

## Types of RDDs

In practice, the Scala API contains an abstract class, `RDD`, which contains not only the five core functions of RDDs, but also those transformations and actions that are available to all RDDs, such as `map` and `collect`. Functions defined only on RDDs of a particular type are defined in several `RDD Functions` classes, including `PairRDDFunctions`, `OrderedRDDFunctions` and `GroupedRDDFunctions`. The additional methods in these classes are made available by implicit conversion from the abstract `RDD` class, based on type information or when a transformation is applied to an RDD.

The Spark API also contains specific implementations of the `RDD` class that define specific behavior by overriding the core properties of the `RDD`. These include the `NewHadoopRDD` class discussed above, which represents an RDD created from an HDFS file system, and `ShuffledRDD`, which represents an RDD, that was already partitioned. Each of these `RDD` implementations contains functionality that is specific to RDDs of that type. Creating an RDD, either through a transformation or from a Spark Context, will return one of these implementations of the `RDD` class. Some `RDD` operations have a different signature in Java than in Scala. These are defined in the `JavaRDD.java` class.

We will discuss the different types of RDDs and RDD transformations in detail in [???](#) and [???](#)

## Functions on RDDs: Transformations vs. Actions

There are two types of functions defined on RDDs, *actions* and *transformations*. Actions are functions that return something that isn't an RDD and transformations that return another RDD.

Each Spark program must contain an action, since actions either bring information back to the driver or write the data to stable storage. Actions that bring data back to the driver include `collect`, `count`, `collectAsMap`, `sample`, `reduce` and `take`.



Some of these actions do not scale well, since they can cause memory errors in the driver. In general, it is best to use actions like `take`, `count`, and `reduce`, which bring back a fixed amount of data to the driver, rather than `collect` or `sample`.

Actions that write to storage include `saveAsTextFile`, `saveAsSequenceFile`, and `saveAsObjectFile`. Actions that save to Hadoop are made available only on RDDs of key-value pairs; they are defined both in the `PairRDDFunctions` class (which provides methods for RDDs of tuple type by implicit conversation) and the `NewHadoopRDD` class, which is an implementation for RDDs that were created by reading from Hadoop. Functions that return nothing (unit type and Scala), such as `foreach`, are also actions and force execution of a Spark job that can be used to write out to other data sources or any other arbitrary action.

Most of the power of the Spark API is in its transformations. Spark transformations are general coarse grained transformations used to sort, reduce, group, sample, filter, and map distributed data. We will talk about transformations in detail in both [???](#), which deals exclusively with transformations on RDDs of key / value data, and [???](#), and we will talk about advanced performance considerations with respect to data transformations.

## Wide vs. Narrow Dependencies

For the purpose of understanding how RDDs are evaluated, the most important thing to know about transformations is that they fall into two categories: transformations with *narrow dependencies* and transformations with *wide dependencies*. The narrow vs. wide distinction has significant implications for the way Spark evaluates a transformation and, consequently, for its performance. We will define narrow and wide transformations for the purpose of understanding Spark's execution paradigm in [“Spark Job Scheduling” on page 28](#) of this chapter, but we will save the longer explanation of the performance considerations associated with them for [???](#).

Conceptually, narrow transformations are operations with dependencies on just one or a known set of partitions in the parent RDD which can be determined at design